3주차 풀이 읽기자료

해설 사용 순서

먼저 학생 실습에서 문제를 독립적으로 풉니다. 이 자료는 힌트와 완전 해설을 포함합니다. 필요한 단계까지만 읽고 다시 풀이합니다.

표시된 예시는 강의에서 만든 예시입니다. 원문과 전체 제약은 각 공식 링크에서 확인합니다.

학생 실습

교사용 진행안

코드 전체 내려받기

74 · 단계별 힌트

LeetCode 74: Search a 2D Matrix · Medium

신규 연습

H1. 평탄 인덱스 4는 어느 행·열인가요?

H2. 행끼리의 순서가 없으면 전체 정렬인가요?

H3. [left,right]를 닫힌 후보 구간으로 두고 mid//C, mid%C로 값을 읽습니다.

74 · 정당성과 비용

행 사이 조건까지 있으므로 평탄 배열은 정렬되어 있습니다. mid가 target보다 작으면 mid 이하를 제거합니다. 매 반복 구간이 줄고, 종료하면 후보가 없습니다.

R,C=행·열 수. O(log(RC)) 시간, O(1) 보조 공간. 평탄 복사하지 않습니다.

74 · 반례와 검증

left=mid는 길이 2 구간에서 정체할 수 있습니다. [[1,9],[2,10]]은 행만 정렬된 다른 문제입니다.

```
matrix=[[1,4,7],[9,12,18]], target=12 → True
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

74 · 구현 1/1

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def search_matrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        if value < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

33 · 단계별 힌트

LeetCode 33: Search in Rotated Sorted Array · Medium

보강

H1. 중간을 기준으로 어느 쪽은 반드시 정렬되어 있나요?

H2. target이 그 값 범위 안에 있나요?

H3. 정렬된 절반을 식별하고 target이 그 범위에 있는지로 남길 절반을 정합니다.

33 · 정당성과 비용

서로 다른 값의 한 번 회전 배열에서는 적어도 한 절반이 정렬되어 있습니다. 그 범위 안/밖 검사는 target을 포함할 수 없는 절반만 제거하며 mid±1로 진전합니다.

n=길이. O(log n) 시간, O(1) 보조 공간.

33 · 반례와 검증

중복을 허용한 [1,0,1,1,1]에서는 양 끝과 mid가 같아 정렬 방향 판정이 모호합니다. 공식 제약 밖 확장입니다.

```
nums=[6,8,1,3,5], target=3 → 3
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

33 · 구현 1/1

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

206 · 단계별 힌트

LeetCode 206: Reverse Linked List · Easy

신규 연습

H1. next를 덮어쓴 뒤 나머지 노드를 어떻게 찾나요?

H2. 처리한 부분과 남은 부분의 head는 무엇인가요?

H3. following에 다음 참조를 보존하고 current.next=previous로 뒤집은 뒤 두 참조를 전진합니다.

206 · 정당성과 비용

반복 시작 때 previous는 처리한 접두부의 역순, current는 미처리 접미부입니다. 한 노드를 옮겨도 둘의 합은 원래 노드 전체입니다. current가 None이면 previous가 완성된 역순입니다.

n=노드 수. O(n) 시간, O(1) 보조 공간. 노드를 재사용합니다.

206 · 반례와 검증

다음 참조를 먼저 저장하지 않으면 나머지 연결을 잃습니다. 같은 값의 두 노드도 서로 다른 객체입니다.

```
head=[4,6,9] → [9,6,4]
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

206 · 구현 1/1

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def reverse_list(head):
    previous, current = None, head
    while current:
        following = current.next
        current.next = previous
        previous, current = current, following
    return previous
```

21 · 단계별 힌트

LeetCode 21: Merge Two Sorted Lists · Easy

신규 연습

H1. 두 head 중 작은 값보다 먼저 올 미처리 값이 있나요?

H2. 결과의 첫 노드를 매번 특별 취급해야 하나요?

H3. dummy 뒤 tail을 유지하고 작은 head를 연결합니다. 한 리스트가 끝나면 나머지를 붙입니다.

21 · 정당성과 비용

tail까지는 정렬된 병합 접두부입니다. 두 head 중 작은 것은 남은 모든 값 중 최소이므로 다음 노드로 안전합니다. 남은 접미부도 정렬되어 있습니다.

n,m=두 노드 수. O(n+m) 시간, O(1) 보조 공간. 입력 연결을 변경합니다.

21 · 반례와 검증

head 대신 dummy를 반환하면 가짜 값이 포함됩니다. 이 구현은 서로 겹치지 않고 사이클 없는 입력 리스트를 전제로 합니다.

```
first=[1,5], second=[2,3] → [1,2,3,5]
```

직접 추적: 실행 전에 각 상태와 반환값을 적고 코드와 비교합니다. 후속 질문: 입력을 바꾸지 않는 조건, 다른 출력 요구 또는 자료구조의 비용이 달라지면 무엇을 수정해야 하나요?

21 · 구현 1/1

아래 구현은 코드 묶음에서도 확인할 수 있습니다.

```
def merge_lists(first, second):
    dummy = tail = ListNode()
    while first and second:
        if first.val <= second.val:
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first or second
    return dummy.next
```

실행과 참고자료

```
cd code
python verify.py
```

공통 객체와 표준 라이브러리 import는 `structures.py`, `solutions.py`에 있습니다. LeetCode에서는 공식 메서드 서명에 맞춰 함수를 연결합니다.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

짧은 확인 문제

1. 열이 네 개인 행렬에서 평탄 인덱스 6은 어느 칸인가요?
2. 연결을 뒤집기 전에 다음 노드를 저장하는 이유는 무엇인가요?
3. 두 입력 리스트가 비어 있으면 `dummy.next`는 무엇인가요?

답: (1) 1행 2열입니다. (2) 저장하지 않으면 현재 노드에서 미처리 부분으로 가는 연결을 잃습니다. (3) `None`입니다.